

Batch A

Foundational Ecological Cases A1-A4 Specification, Implementation & Reporting Plan

An agent-based model of agents that survive a fixed ecology by optimizing among competing physical needs — built as the substrate the lab's humanistic work (Batch B) stands on.

Complete implementation of cases A1-A4 · ODD protocol specification
Parameter table · Experimental design · Ablation controls · Results
Deterministic · config-driven · reproducible from (code, config, seed)

Generated from the project sources, configuration, and measured calibration runs.

Contents

Executive Summary	6
What was built	6
Headline findings (measured)	6
How to read this report	6
Batch A — §9 Decision Record	8
§9 decision points	8
Project-level	8
Consequence accepted (§1)	8
Performance & verification rulings (2026-07-06)	8
Batch A — ODD Protocol Specification	10
1. Overview	10
1.1 Purpose	10
1.2 Entities, state variables, and scales	10
1.3 Process overview and scheduling (§4)	11
2. Design concepts	11
3. Details	12
3.1 Initialization (§5 — the independent variable)	12
3.2 Input data	12
3.3 Submodels	13
3.4 The A3 → A4 phase change (§2.2)	13
3.5 Determinism (§10)	13
Batch A — Parameter Table	14
World & perception (§2)	14
Scalar fields (§2.1)	14
Consumable & threat entities (§2.2)	14
Non-stationarity (§2.2, A4)	15
Needs (§3.1)	15
Actions, decision, learning, traits (§3.3–3.6)	16
Reproduction (§9.4)	16
Initialization, logging, metrics (§5–§7)	17
Implementation Walkthrough	18

Module map	18
Determinism: one seed, separable streams	18
The decision rule and its swappable weight function	19
The tick pipeline (process scheduling)	19
Batch A — Experimental Design	21
Aggregation discipline (§8.3)	21
Operational definitions (§7) — every insight gets a number	21
Per-case design	21
A1 — single-agent learning baseline	21
A2 — population, movement cost, selection	22
A3 — full stationary ecology	22
A4 — the non-stationary phase change	22
Controls / ablations (§8.4)	23
Sensitivity (§8.5)	23
Results & Calibration Narrative	25
A1 — the learning baseline works	25
A2 — population and selection	25
A3 — the full stationary ecology works	25
A4 — fragility, and how it was calibrated	25
Measured A3 → A4 contrast (cap 2000 × 20 seeds, 3000 ticks)	26
How to Run & Reproduce	27
Install	27
Run, sweep, verify	27
Experiments with ablations and cross-seed aggregation	27
Tests	27
Appendix A — Repository File Tree	28
Appendix B — Configuration Listings	30
base.yaml — all defaults	30
a1.yaml	33
a2.yaml	33
a3.yaml	33
a4.yaml	34

ablations/a2_freeze_learning.yaml	35
ablations/a2_freeze_traits.yaml	35

Executive Summary

Batch A is an agent-based model (ABM) of agents that survive a fixed ecology by optimizing among competing physical needs. It is deliberately *not* the interesting science — it is the **substrate** the lab's humanistic work (Batch B: attention economics, social needs, grief, sacrifice) stands on. Its job is to produce agents that (a) demonstrably learn / are selected to exploit their environment, and (b) survive it **poorly enough**, by case A4, that Batch B's mechanisms have real pressure to act on.

These two criteria pull against each other, and the governing discipline of this work is that **both are measured, not asserted**. This report is the measurement.

What was built

A complete, deterministic, config-driven implementation of the four foundational cases, with the module seams chosen now so Batch B drops in without rewrites:

- **A1** — single-agent learning baseline (moisture field + water). *Does an agent learn to exploit the environment?*
- **A2** — population, movement cost, birth/death → **selection**. *Does birthplace × disposition predict survival?*
- **A3** — full **stationary** ecology: heat (a comfort band), vegetation, prey, predators (risk). *Does it still work with the whole need set?*
- **A4** — the **non-stationary phase change**: seasonality, mobile fauna, drought/rebound. *Survival turns fragile*.

Headline findings (measured)

- **A1 works**. The single agent learns to navigate the moisture gradient to water and survives every seed tested (6/6). Decoupling moisture from water kills it by ~tick 100 — confirming the learned navigation is real, not luck.
- **A2 selects**. Reproduction yields hundreds of births per run; mortality is structured (movement-cost energy and thirst), and the heritable exploration disposition is exposed to selection.
- **A3 works, uniformly**. The population stabilizes near carrying capacity (mean ≈ 360–383) in essentially every seed (0 of 2–3 extinct).
- **A4 is fragile**. Under non-stationarity the same population becomes precarious: it persists in some worlds and collapses in others (≈ 3/5 extinct at a 2500-tick horizon), with violent oscillation, deep recurrent crashes, and roughly half the individual survival of A3. This is "survive it poorly enough that Batch B has something to do," made quantitative.

How to read this report

The report is self-contained and ordered so a stranger could rebuild the model: the **Decision Record** fixes the modeling choices; the **ODD Specification** describes the model to the field's standard (Grimm/Railsback); the **Parameter Table** lists every constant with units; the **Implementation Walkthrough** ties the spec to the code; the **Experimental Design** and **Results** sections give the measurement protocol and the numbers; and the **Appendices** carry the full file tree and configuration listings.

Batch A — §9 Decision Record

The spec (§9) flagged five modeling decisions where defaults were assumed and a ruling was needed, plus the project-level questions of scope and location. This records what was ruled and how it is realized in code.

§9 decision points

#	Decision (§)	Ruling	Where it lives
1	Learning + traits both present, or A1 purely scripted? (§1)	Both present. A1 runs as a single-agent <i>learning</i> baseline with traits fixed (<code>traits.fix_identical: true</code>); A2+ add population, traits, and selection.	<code>agent/learning.py</code> , <code>agent/traits.py</code> ; <code>config/a1.yaml</code>
2	Continuous fields + discrete entities, or discrete tiles only? (§2.1)	Continuous scalar fields + discrete entities. Moisture/temperature/risk are scalar fields perceived as gradients; water/vegetation/prey/predators are discrete.	<code>environment/fields.py</code> , <code>environment/entities.py</code>
3	Synchronous vs. sequential updating? (§4)	Synchronous snapshot updating — no first-mover advantage, reproducible. Flagged because A2's movement-cost result can be sensitive to it.	<code>scheduler.py</code>
4	Reproduction model for A2+ (§9.4)	Asexual clone-with-mutation. Simplest model that still yields selection; the action registry + reproduction seam leave room for Batch B's mating.	<code>reproduction.py</code> , <code>agent/traits.py</code>
5	How harsh is A4? (§9.5)	Fragile survival — the population persists but poorly, leaving pressure for Batch B. Not near-total collapse. All harshness knobs are config in <code>config/a4.yaml</code> and were calibrated empirically (see <code>EXPERIMENTAL_DESIGN.md</code>).	<code>config/a4.yaml</code> , <code>environment/dynamics.py</code>

Project-level

Question	Ruling
Scope of this pass	Full A1-A4 implementation , not just A1.
Location	A self-contained Python project in <code>batch-a/</code> inside the existing repo, on the working branch. Can be split into its own repo later without code changes.
Language / deps	Python 3.11; numpy + PyYAML. CSV logging always available; Parquet used opportunistically if pandas is importable.

Consequence accepted (§1)

With both learning and traits present, "explorative agents survived" is ambiguous — trait or learned policy? Batch A's rigor depends on disentangling these by ablation, which is why `freeze_learning` and `freeze_traits` are first-class config overlays and the experiment driver runs them as controls. If neither channel alone reproduces a result, that result is an **interaction** — reported as such, not papered over.

Performance & verification rulings (2026-07-06)

Question	Ruling
Fast path scope	Vectorize perception only (<code>perception.impl: vectorized</code> , now default). Decision, RNG draws, and conflict resolution stay on the scalar reference — their conditional per-agent draws cannot be batched without moving the reference. Proven bit-identical at the full 3000-tick horizon on seam-straddling wrap, saturated-tie, rich, and poor worlds (<code>experiments/verify_perception.py</code>); guarded at field level by <code>tests/test_perception_equivalence.py</code> .
What bit-identity means	No-regression, not correctness. Both impls descend from the same source and can agree while both wrong. A green diff licenses swapping the fast path in; it never validates the model.
The deferred 10×	Struct-of-arrays rewrite is design-frozen (<code>docs/SOA_REWRITE_PLAN.md</code>): it necessarily moves the RNG-consumption reference (R1→R2 via counter-based keying), so it waits until the current result set is closed and archived. Enabling tooling (RNG trace/replay, rung 3) is built and tested (<code>experiments/rng_trace.py</code>).
Code freeze during headline runs	<code>batch_a/</code> sim internals are frozen while a full-N grind is in flight. Cells must not span two code versions; each checkpoint row records its <code>impl</code> for provenance.
Estimator of record for WO-2 β_s	Crash-robust stable-tick hazard (grouped Poisson over stable ticks; Cox-on-age cross-check), fitted inline per cell; raw OLS β_s demoted to sensitivity. Validated on the pilot (7/7 sign agreement) and on planted-sign synthetic data (<code>tests/test_wo_instruments.py</code>).
Headline moderator question	Decided by the full-20's disentangling regression $\beta_s \sim z(\text{volatility}) + z(\text{capacity})$ with seed-bootstrap CIs; a moderator wins only if its CI excludes zero (<code>experiments/analyze_wo2_full20.py</code>).
One palette source	<code>design/tokens.json</code> → figures (<code>viz/style.py</code>) AND site CSS (<code>design/build_css.py</code> → <code>src/app/tokens.css</code>); drift is test-guarded.

Batch A — ODD Protocol Specification

This document follows the **ODD protocol** (Overview, Design concepts, Details; Grimm & Railsback) — the field standard for describing an ABM so a stranger can rebuild it. It is the difference between a writeup someone can replicate and one they can't. Section references (§) point to the Batch A spec.

*Conventions. The world is a size $\times$ size grid. Positions are integer tile coordinates (x, y) ; arrays are indexed $[y, x]$. Boundaries are **toroidal** (wrap) for A1–A4 unless reconfigured. All needs and field values live in $[0, 1]$. Every numeric constant cited here is a **config value** — see `PARAMETERS.md` for the table and `config/` for the source of truth.*

1. Overview

1.1 Purpose

To produce agents that (a) **survive a fixed ecology by optimizing among competing physical needs**, and (b) survive it **poorly enough by A4** that the humanistic mechanisms of Batch B have pressure to act on. Batch A exists to be *measured against two criteria that pull against each other*: it must work (agents learn/are selected to exploit the world) and it must not be enough (survival is fragile under non-stationarity). Neither criterion is asserted; both are logged and quantified (§0, §6, §7).

1.2 Entities, state variables, and scales

Agent. The only mobile decision-maker. State:

Variable	Meaning
id , (x, y)	identity and tile position
state (need vector)	$S_i(t) \in [0,1]$ for each need, with target T_i and threat threshold $L_i < T_i$
traits	heritable vector; Batch A's load-bearing trait is exploration disposition $\in [0,1]$
learner	per-(need, action) learned gains — the within-lifetime $Q_i(a)$ (§3.5)
memory	stub in A; the seam for Batch B's distorting memory (§10)
age , birth_tick , repro_cooldown	reproduction bookkeeping
offspring_count , cause_of_death , birthplace	lifetime outcomes (§6)

The **need vector is ordered and extensible**. Batch A ships `energy` , `hydration` , `temperature_comfort` , `safety` ; Batch B appends higher needs onto this exact structure, so no code assumes the Batch A need set.

Environment. Built once and layered A1→A4 (§2):

- *Continuous scalar fields* (§2.1): `moisture` , `temperature` , `risk` . Agents

perceive these as **local gradients**, never as a global map.

- *Discrete consumable fields*: `water` , `vegetation` — quantity on a per-tile grid sitting on a fixed clustered **source mask**, regenerating toward a cap.
- *Discrete populations*: `prey` (eatable points) and `predators` (threats that emit the risk field and can kill within an attack radius).

Scales. One tick = one decision/update cycle. The grid starts 64×64 for fast iteration and is parameterizable up to 256×256 (the paper's size) — size is a config value, never hard-coded. Perception is a bounded Moore radius $r \in \{1, 2\}$.

1.3 Process overview and scheduling (§4)

Update order is **load-bearing** and never drifts. Each tick:

1. **Environment update** — regeneration; in A4 also: season tick, fauna move, drought roll.
2. **All agents perceive** — on a *snapshot*, so everyone sees the same world.
3. **All agents choose actions.**
4. **Actions resolve** — consumption, movement, predation. Conflict resolution (two agents, one source) is defined here (see §2.2 below).
5. **State drains applied** — endogenous drain, exposure, movement cost.
6. **Learning updates** — $Q_i(a)$ adjusts toward realized outcomes.
7. **Death checks** (state ≤ 0 or predation), then **births** (A2+).
8. **Logging.**

Updating is **synchronous** (steps 2–3 act on the pre-resolution snapshot): no first-mover advantage, and runs are reproducible. (Sequential random-order updating is the documented alternative; A2's movement-cost result can be sensitive to this, so the choice is explicit, §4.)

2. Design concepts

Basic principles. Homeostatic control: each agent keeps a vector of needs near target by choosing actions that restore the most pressing deficits, net of movement cost and risk. Adaptation has two channels, deliberately separable for ablation (§1 of the spec):

- **Learned** — within one lifetime, $Q_i(a)$ updates (recency / TD(0)).
- **Selected** — across generations, heritable traits + birth/death.

Objectives (decision rule, §3.4). Each agent scores actions by

$$U(a) = \sum_i w_i(t) \cdot Q_i(a) - \text{MoveCost}(a) - \text{Risk}(a)$$

and chooses argmax with ϵ -stochasticity. $w_i(t)$ is deficit-proportional in Batch A and is computed behind a **single swappable function** (`compute_weights`) so Batch B can drop in Maslow-banded gating without touching anything else. $Q_i(a) = \text{learned_gain}[i][a] \times \text{perceptual_affordance}[i][a]$: the affordance is the current perceptual signal (e.g. water available locally), the gain is what the learner adapts.

Learning. $\text{gain}[i][a] \leftarrow \text{gain}[i][a] + \alpha \cdot (\text{realized/expected} - \text{gain}[i][a])$ (recency); TD(0) adds a $\gamma \cdot V(\text{next})$ bootstrap. One config flag disables learning for the ablation — gains then stay at their prior and the perceptual policy still runs, which is exactly what isolates the trait channel.

Prediction & sensing. Agents sense only their Moore neighbourhood: field gradients and entity presence within radius r . Moisture is generated to **peak on water sources**, so "ascend the moisture gradient" genuinely leads to water — the spec's "paths of increasing moisture". A faint-but-correct gradient still commits the agent to a heading (a config floor), enabling long-range navigation.

Interaction. Agents compete for finite consumable tiles. Conflict resolution (§4 step 4): claims are grouped by tile, then satisfied in a **seed-deterministic shuffled order** until the tile depletes. Predators interact with agents by emitting risk and attacking within their attack radius.

Stochasticity. ϵ -stochastic action choice and random-step exploration noise, both scaled by the exploration trait; spawn placement; mutation; environment rolls (drought, fauna movement). Every draw comes from an explicit, seeded stream (§ Details/Initialization).

Collectives. None in Batch A (asexual, non-social). The action registry and reproduction seam leave room for Batch B's social actions and mating.

Emergence vs. imposed. Survival, settling, over-exploration, and the selected trait distribution are **emergent** and measured (§7). The ecology, update order, and need dynamics are **imposed**.

Observation (§6, §7). Logged from tick 1: per-tick per-agent state; cause of death; per-agent lifetime summaries (birthplace endowment, time-to-settle, fraction stationary, over-exploration distance, survival time, offspring count); per-run seed, full config, and environment hash. Operational definitions turn each headline insight into a number — see EXPERIMENTAL_DESIGN.md §Operational.

3. Details

3.1 Initialization (§5 — the independent variable)

Birthplace is a **controlled, logged condition, not incidental randomness**. At $t=0$:

- The world is built from the environment RNG stream (fields, source masks, prey/predator placement).
- Founders are placed by the **spawn** RNG stream, separable from the environment stream — so the world can be held fixed while only spawns vary (or vice versa).
- Spawn mode is one of `uniform`, `born_rich`, `born_poor`. `born_rich` draws founder tiles from the top resource-endowment percentile; `born_poor` from the bottom.

Resource endowment at birth = mean of the per-tile endowment (normalized moisture + water + vegetation) within radius r of the spawn tile. It is logged per agent and is the scalar behind "born into resources".

- Founder traits are drawn from $N(\text{init_mean}, \text{init_sd})$ (clipped to $[0,1]$), or fixed identical under the trait-frozen ablation.

3.2 Input data

None external. The world is generated procedurally from config + seed; there are no time-series driver files. (A4 non-stationarity is generated internally.)

3.3 Submodels

Need dynamics (§3.1, §4 step 5). Each tick every need drains by its `drain_per_tick`. Two needs additionally respond to exposure:

- `temperature_comfort` : outside the comfort band it drains by $\text{exposure_gain} \times \text{distance_outside_band}$; inside it recovers by `recovery` .
- `safety` : drains by $\text{exposure_gain} \times \text{local_risk}$; recovers when risk is low.

Actions restore needs in step 4 (before drains): `drink` → hydration from the water field; `eat` → energy from prey (preferred) or vegetation; `rest` → energy by a fixed amount; movement pays `move_cost_per_tile` from energy (A2+).

Action set (§3.3). A **registry**, not an enum: `move` , `drink` , `eat` , `rest` , `flee` . When `move` is chosen, direction is a weighted vote over per-need perceptual cues; `flee` steps down the risk gradient. The exploration trait injects ϵ action-noise and random-step noise — the mechanism behind "explores forever even past optimal" (§7).

Reproduction (§9.4). Asexual clone-with-mutation. A parent that is mature, off cooldown, has all needs $\geq \text{readiness_min_need}$, and enough energy, spends `energy_cost` to spawn one offspring within `offspring_spawn_radius` . The offspring inherits the parent's traits with gaussian mutation (or identical under ablation). A `max_population` cap keeps runs bounded.

Mortality (§6). Death occurs when any need hits 0 (cause = that need) or by predation (cause = `predation`). Cause-of-death accounting exists from the start because Batch B's "death by grief / sacrifice" requires it.

3.4 The A3 → A4 phase change (§2.2)

A1–A3 are **stationary**. A4 toggles non-stationarity on the *same* world (a config delta, not a fork):

- **Seasonality** — the temperature field oscillates by `season_amplitude` over `season_period` ; vegetation is `season_gated` (edible only part of the year).
- **Mobile fauna** — prey and predators step with per-tick probability; the risk field follows the predators.
- **Drought/rebound** — water sources stochastically enter drought (regen scaled by `dry_regen_factor`) and recover, governed by `onset_prob` / `recovery_prob` .

Everything an agent learned or was selected for in A3 can be invalidated in A4. **That is the point.** Harshness is tuned to *fragile survival* (the population persists but poorly), per the §9.5 ruling; all harshness knobs live in `config/a4.yaml` .

3.5 Determinism (§10)

A single master seed per run seeds a `SeedSequence` , spawning four independent generators: **environment**, **spawn**, **agent** (decision/conflict), and **mutation**. The global RNG is never touched. Two runs with the same (code, config, seed) produce identical histories; `python -m batch_a verify` asserts this, and `tests/test_determinism.py` guards it in CI.

Batch A — Parameter Table

Every constant, its value, and its units (§8.2). **No magic numbers live in prose or code** — `config/base.yaml` is the source of truth; per-case configs override only the deltas noted in the last column. Values below are the base defaults.

Units: frac = a fraction in [0,1] (need levels, field values); tiles = grid cells; ticks = simulation steps; prob = per-event probability.

World & perception (§2)

Parameter	Value	Units	Meaning	Case overrides
<code>world.size</code>	64	tiles	grid is size×size (up to 256 for final runs)	—
<code>world.boundary</code>	toroidal	—	wrap vs. walls	—
<code>world.perception_radius (r)</code>	2	tiles	bounded Moore radius; local info only	—
<code>perception.gradient_cue_gain</code>	8.0	—	scales gradient magnitude → cue strength	—
<code>perception.gradient_cue_floor</code>	0.35	frac	min cue strength when a gradient direction exists	—

Scalar fields (§2.1)

Parameter	Value	Units	Meaning	Introduced
<code>fields.moisture.clusters</code>	5	count	wet clusters (when not colocated with water)	A1
<code>fields.moisture.cluster_sigma</code>	6.0	tiles	spread of each moisture cluster	A1
<code>fields.moisture.baseline</code>	0.05	frac	field floor	A1
<code>fields.temperature.gradient_axis</code>	y	—	axis of the linear temperature gradient	A3
<code>fields.temperature.low / .high</code>	0.0 / 1.0	frac	field values at the two edges	A3
<code>fields.temperature.comfort_low / .high</code>	0.25 / 0.75	frac	comfort band bounds	A3
<code>fields.temperature.season_amplitude</code>	0.0 (A4: 0.05)	frac	peak seasonal offset added to the field	A4
<code>fields.temperature.season_period</code>	365 (A4: 400)	ticks	ticks per seasonal cycle	A4
<code>fields.risk.predator_sigma</code>	4.0	tiles	spread of each predator's risk halo	A3
<code>fields.risk.baseline</code>	0.0	frac	ambient risk floor	A3

Consumable & threat entities (§2.2)

Parameter	Value	Units	Meaning	Introduced
entities.water.sources	6	count	water source clusters (also seed the moisture peaks)	A1
entities.water.cluster_sigma	2.0	tiles	source core spread	A1
entities.water.capacity	1.0	frac	max stored quantity per source tile	A1
entities.water.regen_per_tick	0.05	frac/tick	restored toward capacity	A1
entities.water.efficiency	1.0	—	consumed quantity → hydration gain	A1
entities.vegetation.patches	8	count	vegetation patches	A3
entities.vegetation.cluster_sigma	3.0	tiles	patch spread	A3
entities.vegetation.regen_per_tick	0.02 (A4: 0.03)	frac/tick	regrowth toward capacity	A3
entities.vegetation.efficiency	0.8	—	consumed quantity → energy gain	A3
entities.vegetation.season_gated	false (A4: true)	—	edible only in-season	A4
entities.prey.count	12	count	discrete prey	A3
entities.prey.respawn_ticks	80	ticks	delay before a consumed prey respawns	A3
entities.prey.efficiency	1.0	—	eating prey → energy gain	A3
entities.prey.move_prob	0.0 (A4: 0.3)	prob	per-tick step probability	A4
entities.predators.count	6 (A4: 4)	count	discrete predators	A3
entities.predators.attack_radius	1	tiles	predation range (Chebyshev)	A3
entities.predators.predation_prob	0.15	prob	per-tick kill probability in range	A3
entities.predators.move_prob	0.0 (A4: 0.15)	prob	per-tick step probability	A4

Non-stationarity (§2.2, A4)

Parameter	Value (A4)	Units	Meaning
nonstationarity.seasonality	true	—	temperature oscillates; vegetation season-gated
nonstationarity.mobile_fauna	true	—	prey & predators move
nonstationarity.drought.onset_prob	0.0018	prob	wet→drought flip per tick
nonstationarity.drought.recovery_prob	0.012	prob	drought→wet flip per tick
nonstationarity.drought.dry_regen_factor	0.30	—	water regen multiplier during drought

Needs (§3.1)

Need	initial	target	threshold	drain/tick	exposure_gain	recovery
energy	1.0	1.0	0.25	0.004	—	—
hydration	1.0	1.0	0.25	0.010	—	—
temperature_comfort	1.0	1.0	0.20	0.000	0.15	0.05
safety	1.0	1.0	0.20	0.000	0.50	0.05

All in *frac* (levels) or *frac/tick* (rates). Exposure drain = exposure_gain × signal (comfort: distance outside band; safety: local risk).

Actions, decision, learning, traits (§3.3–3.6)

Parameter	Value	Units	Meaning	Case overrides
actions.rest.amount	0.06	frac/tick	energy restored by resting	—
actions.move_cost_per_tile	0.0 (A2+: 0.012)	frac/tile	energy drained per tile moved	A2–A4
actions.flee_cost_per_tile	0.0 (A2+: 0.012)	frac/tile	energy drained per tile fled	A2–A4
decision.weighting	deficit_proportional	—	the single swappable weight function	—
decision.epsilon	0.05	prob	base ϵ for ϵ -stochastic choice	—
decision.risk_weight	1.0	—	coefficient on Risk(a) in U(a)	—
decision.move_cost_weight	1.0	—	coefficient on MoveCost(a) in U(a)	—
learning.enabled	true	—	within-lifetime learning on/off (ablation)	freeze_learning
learning.rule	recency	—	recency (delta rule) or td0	—
learning.alpha	0.2	—	learning rate	—
learning.gamma	0.9	—	discount (td0 only)	—
learning.gain_init	1.0	—	initial learned gain per (need, action)	—
learning.gain_min / .gain_max	0.1 / 3.0	—	gain clamp	—
traits.fix_identical	false (A1: true)	—	fix all dispositions identical (ablation)	A1, freeze_traits
traits.exploration.init_mean	0.5	frac	mean founder disposition	—
traits.exploration.init_sd	0.15	frac	sd of founder disposition	—
traits.exploration.mutation_sd	0.05	frac	gaussian mutation sd to offspring	—
traits.exploration.epsilon_scale	0.30	—	$\epsilon \ += \text{exploration} \times \text{this}$	—
traits.exploration.step_noise_scale	0.50	prob	random-step probability = exploration × this	—

Reproduction (§9.4)

Parameter	Value	Units	Meaning
reproduction.enabled	false (A2+: true)	—	asexual clone-with-mutation
reproduction.maturity_age	40	ticks	minimum age to reproduce
reproduction.cooldown	25	ticks	ticks between births for one agent
reproduction.readiness_min_need	0.6	frac	all needs must be $\geq$ this to reproduce
reproduction.energy_cost	0.4	frac	energy spent per offspring
reproduction.offspring_spawn_radius	1	tiles	offspring placement radius around parent
reproduction.max_population	400	count	hard population cap

Initialization, logging, metrics (§5-§7)

Parameter	Value	Units	Meaning
init.n_agents	1 (A2/A3: 40, A4: 60)	count	founder population
init.spawn_mode	uniform	—	uniform / born_rich / born_poor
init.endowment_percentile	0.85	frac	rich/poor spawn cutoff
init.endowment_radius	2	tiles	radius for the birthplace endowment scalar
logging.format	csv	—	csv (always) or parquet (needs pandas)
logging.out_dir	runs	—	runs/<run_id>/ per run
metrics.settle_displacement_threshold	1.5	tiles	below this counts as "settled"
metrics.settle_window	50	ticks	window displacement must stay low to settle
metrics.resource_percentile_k	0.75	frac	Kth percentile defining a "good" tile
run.max_ticks	per case: A1 1500, A2/A3 3000, A4 2500	ticks	run horizon (also ends on extinction)

Implementation Walkthrough

This part ties the specification to the code. The repository is a self-contained Python project (`batch-a/`) with the module boundaries the spec's §10 prescribes, chosen so that Batch B is a set of drop-ins rather than a rewrite. Everything is config-driven (no magic numbers in code) and deterministic.

Module map

Module	Responsibility	Batch-B seam
<code>rng.py</code>	one seeded master → separable streams (env / spawn / agent / mutation)	—
<code>config.py</code>	<code>base.yaml</code> ← case ← ablation deep-merge; environment hash	—
<code>environment/fields.py</code>	continuous scalar fields, toroidal gradients	—
<code>environment/entities.py</code>	discrete consumables + prey/predator populations	—
<code>environment/world.py</code>	the shared substrate, layered A1→A4	non-stationarity is a toggle
<code>environment/dynamics.py</code>	per-tick environment update (incl. A4)	A4 is a subclass of behaviour
<code>agent/state.py</code>	extensible homeostatic need vector	B appends higher needs
<code>agent/perception.py</code>	local gradients + entity presence, radius r	—
<code>agent/actions.py</code>	action registry , not an enum	B adds share/signal/mate
<code>agent/decision.py</code>	$U(a) = \sum w_i \cdot Q_i(a) - \text{cost} - \text{risk}$; weights behind one function	Maslow gating drops in
<code>agent/learning.py</code>	within-lifetime $Q_i(a)$ update; one flag disables it	—
<code>agent/traits.py</code>	heritable exploration disposition + mutation	—
<code>agent/memory.py</code>	stub seam	B's distorting memory plugs in
<code>scheduler.py</code>	the fixed 8-step tick pipeline (synchronous)	—
<code>reproduction.py</code>	asexual clone-with-mutation (A2+)	B adds mating
<code>metrics.py</code>	§7 operational definitions + bootstrap aggregation	—
<code>logging/</code>	§6 observable schema + run recorder	—
<code>sim.py / cli.py</code>	build world, run, report; run / sweep / verify	—

Determinism: one seed, separable streams

Determinism is non-negotiable: a run is reproducible from (`code`, `config`, `seed`) alone. A single master seed seeds a NumPy `SeedSequence` , which *spawns* four independent generators. The global RNG is never touched, and because the streams are separate, the world can be held fixed while only spawns vary (or vice versa) — exactly what the §5 birthplace experiments need.

```
class RNGStreams:
    """Bundle of independent numpy Generators derived from one master seed.

    Attributes
```

```

-----
environment : world dynamics (regeneration, fauna movement, drought rolls)
spawn       : agent spawn placement (the $5 independent variable)
agent       : agent decision noise ( $\epsilon$ -stochasticity, conflict shuffles)
mutation    : heritable-trait mutation at reproduction
"""

def __init__(self, seed: int):
    self.seed = int(seed)
    children = np.random.SeedSequence(self.seed).spawn(len(_STREAMS))
    for name, child in zip(_STREAMS, children):
        setattr(self, name, np.random.default_rng(child))

```

The decision rule and its swappable weight function

Each agent scores actions by $U(a) = \sum_i w_i(t) \cdot Q_i(a) - \text{MoveCost}(a) - \text{Risk}(a)$ and chooses argmax with ϵ -stochasticity. The need weights $w_i(t)$ are computed behind a **single function** — Batch A uses simple deficit-proportional weighting; Batch B replaces only this function with Maslow-banded gating.

```

def compute_weights(needs, decision_cfg) -> dict:
    """Deficit-proportional need weights  $w_i(t)$ , normalized to sum 1."""
    deficits = needs.deficits()
    total = sum(deficits.values())
    if total <= 1e-9:
        n = len(deficits)
        return {k: 1.0 / n for k in deficits}
    return {k: v / total for k, v in deficits.items()}

```

$Q_i(a) = \text{learned_gain} \cdot \text{perceptual_affordance}$: the affordance is the current perceptual signal (e.g. water available locally), and the gain is what the learner adapts. Freezing learning leaves the perceptual policy intact — which is exactly what isolates the trait channel in the ablation.

```

def update(self, need, action, expected, realized, next_value=0.0):
    """Adapt the (need, action) gain toward the realized outcome.

    ``expected`` is the affordance the agent acted on; ``realized`` is the
    actual need gain obtained. With  $\text{td0}$  the target adds a discounted
    estimate of the next-state value for this (need, action) channel.
    """
    if not self.enabled or expected <= 1e-9:
        return
    target_ratio = realized / expected
    if self.rule == "td0":
        target_ratio += self.gamma * next_value
    key = (need, action)
    g = self._gain[key]
    g += self.alpha * (target_ratio - g)
    self._gain[key] = min(self.gain_max, max(self.gain_min, g))

```

The tick pipeline (process scheduling)

Update order is load-bearing and never drifts. The scheduler implements the 8-step synchronous pipeline; perception and choice act on a pre-resolution snapshot so there is no first-mover advantage, and conflict resolution (two agents, one source) is defined here, once.

```

def run_tick(world, agents, config, rng, recorder):
    tick = world.tick # tick index this step operates on (pre-increment)

    # 1. environment update
    world.update()

    living = [a for a in agents if a.alive]

```

```

prev_pos = {a.id: (a.x, a.y) for a in living}

# 2. perceive (snapshot) – scalar reference, or the bit-identical batch path
if config.perception.get("impl", "scalar") == "vectorized":
    from .agent.perception import perceive_all
    perceptions = perceive_all(world, living, config)
else:
    perceptions = {a.id: a.perceive(world) for a in living}

# 3. choose
decisions = {a.id: a.choose(world, perceptions[a.id], rng.agent)
              for a in living}

# 4. resolve actions
realized, predated = _resolve_actions(world, living, decisions, config, rng)

# 5. drains + per-agent bookkeeping/metrics
_apply_drains(world, living, decisions, config)
for a in living:
    a.age += 1
    if a.repro_cooldown > 0:
        a.repro_cooldown -= 1
    a.metrics.update(prev_pos[a.id], (a.x, a.y), world, a.age)

# 6. learning
for a in living:
    a.learn(realized.get(a.id, 0.0))
    a.memory.observe(a, perceptions[a.id], world)

# 7. deaths, then births
deaths_by_cause: dict[str, int] = {}
for a in living:
    if a.id in predated:
        a.alive = False
        a.cause_of_death = "predation"
    else:
        dead_need = a.state.first_depleted()
        if dead_need is not None:
            a.alive = False
            a.cause_of_death = dead_need
    if not a.alive:
        recorder.log_summary(a)
        deaths_by_cause[a.cause_of_death] = deaths_by_cause.get(a.cause_of_death, 0) + 1
recorder.log_deaths(tick, deaths_by_cause)

newborns = reproduce(agents, world, config, rng, tick)
agents.extend(newborns)

# 8. logging
recorder.log_env(tick, world.env_summary())
recorder.log_fauna(tick, world)
for a in living:
    recorder.log_tick(tick, a)
for a in newborns:
    recorder.log_tick(tick, a)

return len([a for a in agents if a.alive])

```

Batch A — Experimental Design

What "fully reporting on Batch A" requires (§8): an ODD spec (`ODD.md`), a parameter table (`PARAMETERS.md`), and — here — the experimental design: for each case what is varied, what is held fixed, how many seeds, how results are aggregated, the controls/ablations, sensitivity, and how results tie to operational definitions rather than narration.

Aggregation discipline (§8.3)

- **Seeds.** Each reported cell is run across a seed sweep (≥ 20 for headline results; the examples below use fewer for turnaround). Seed controls the master RNG; environment and spawn streams are separable so the world can be held fixed while only spawns vary.
- **Aggregate.** Median across seeds with a **percentile bootstrap CI** (`metrics.bootstrap_ci` , 2000 resamples, 95%). Extinction is reported as a rate.
- **Reproducibility.** Every run records `(git\_commit, config, seed, environment\_hash)`; a result is rebuildable from those alone.

Driver:

```
python experiments/run_experiment.py --config <case>.yaml --seeds 0-19 \  
  --conditions uniform,born_rich,born_poor \  
  --ablations none,freeze_learning,freeze_traits
```

Operational definitions (§7) — every insight gets a number

Narrative claim	Operational measure	Column
conservative ↔ explorative	heritable exploration trait (continuous) + behavioural readout	<code>trait_exploration</code> , <code>mean_step_displacement</code>
"born into resources"	birthplace endowment scalar (§5)	<code>birthplace_endowment</code>
"settles and stays forever"	time-to-settle + fraction of lifetime stationary	<code>time_to_settle</code> , <code>fraction_stationary</code>
"explores forever past optimal"	distance travelled after first reaching a Kth-percentile tile	<code>over_exploration_distance</code>
survival outcome	survival time + reproductive success	<code>survival_time</code> , <code>offspring_count</code>

The A2 claim becomes a testable surface: `survival ~ birthplace_endowment × exploration_trait` , fit across seeds, with the learning-frozen and trait-frozen ablations as controls.

Per-case design

A1 — single-agent learning baseline

- **Varied:** seed (world + agent trajectory). Optionally `learning.enabled` to contrast learning vs. scripted policy.

- **Fixed:** single agent, traits identical, only moisture + water present.
- **Success criterion ("it works"):** the agent learns to exploit water and survives the full horizon, rather than dying of thirst by random walk.
- **Result (seeds 0-5, 1500 ticks):** survives 6/6 seeds to the horizon; cause-of-death empty. The learner's `hydration:drink` gain rises above its prior — learning, not luck. (With moisture and water *decoupled*, the agent dies ~tick 100; colocation + a gradient-cue floor is what makes navigation work — documented as the key A1 modeling fix.)

A2 — population, movement cost, selection

- **Varied:** spawn condition {uniform, born_rich, born_poor} ; ablation {none, freeze_learning, freeze_traits} ; seed.
- **Fixed:** the world (same environment hash across conditions), movement cost, reproduction parameters.
- **Questions:** Does birthplace endowment predict survival/reproduction? Does the exploration trait? Is the effect trait, learning, or their interaction?
- **Reads:** `survival_time` , `offspring_count` , `final_population` by condition; selected `trait_exploration` distribution of survivors vs. founders.

A3 — full stationary ecology

- **Varied:** spawn condition; seed. (Ablations available.)
- **Fixed:** everything stationary — fauna don't move, no seasons, no drought.
- **Success criterion ("it works"):** the population is self-sustaining under the full need set (energy, hydration, comfort, safety) across most seeds.
- **Result (calibration checks, 2500 ticks):** 0/2–0/3 seeds extinct; the population stabilises near carrying capacity (mean ~360–383) — uniformly stable, in deliberate contrast to A4 (see the A3→A4 table below).
- **Mortality structure:** logged by cause (`deaths_by_cause`) — comfort-band exposure and predation are the dominant pressures here.

A4 — the non-stationary phase change

- **Varied:** seed; the harshness knobs in sensitivity analysis.
- **Fixed:** the A3 world generator; A4 only *toggles* seasonality, mobile fauna, and drought on top.
- **Success criterion ("it is not enough"):** survival is **fragile** — the population persists but markedly worse than A3 (lower final population, shorter median survival, higher extinction rate). Calibrated to fragility, not collapse, per the §9.5 ruling.

A3 → A4 contrast (calibration run)

Measured at the 2500-tick horizon (these are turnaround checks at $n = 2-5$ seeds; headline numbers warrant ≥ 20 seeds per the discipline above — the small- n rates here are seed-noisy by ± 1 seed):

Metric	A3 (stationary)	A4 (non-stationary)
extinct by horizon	0 / 2-3 seeds	~3 / 5 seeds

Metric	A3 (stationary)	A4 (non-stationary)
mean population over run	~360-383 (near cap)	50-379, typ. ~300 then crash
min population over run	~30-38	1-58 (teeters at the edge)
median individual survival	176-502 ticks	113-175 ticks (high turnover)
population trajectory	rises and stabilises near carrying capacity	violent oscillation ; deep recurrent crashes

Reading. A3 is uniformly stable: every seed settles into the comfortable temperature refuge and sustains near carrying capacity. A4 is **fragile**: surviving seeds persist *precariously* — substantial populations that crash deeply (to a handful of agents) and claw back, repeatedly — while a sizeable fraction of seeds collapse to extinction outright. Individuals live roughly half as long (constant churn), and the population teeters near zero throughout. This is "survive it poorly enough that Batch B has something to do" made quantitative: the physical-needs substrate **alone** cannot reliably survive non-stationarity.

Why fragility is horizon-dependent (an honest caveat). Extinction is an absorbing state, and a deeply-oscillating population eventually hits zero given enough time — so A4's extinction *rate rises with the reporting horizon*. We fix the horizon at 2500 ticks, where A4 "survives poorly but not totally"; extending it drives more seeds extinct. That horizon-dependence **is** the fragility, not an artefact of it.

The collapse boundary (sensitivity, §8.5). A4 sits near a tipping point. The pre-calibration settings (`season_amplitude` 0.15-0.30, severe drought, 6 fast mobile predators) produced **uniform collapse** (3/3 extinct, whole-population die-offs around the first cold season). The calibrated knobs in `config/a4.yaml` — shallower/shorter seasons, a wider comfort refuge, calmer predators, and faster reproductive recovery — move the system from *guaranteed collapse* to *precarious persistence*. The narrowness of that band is itself a finding, and every knob is config, so the harshness is a dial, not a constant.

Controls / ablations (§8.4)

At minimum, every A2/A3 headline is reported with:

- **freeze_learning** — `learning.enabled: false` . Gains stay at prior; only the heritable disposition adapts. Does the **trait alone** reproduce the pattern?
- **freeze_traits** — `traits.fix_identical: true` . Everyone shares one disposition; only within-lifetime learning adapts. Does **learning alone** reproduce it?

Without these, A2/A3 are stories. If neither alone reproduces a result, it is an **interaction** — a stronger and more honest finding (§1).

Sensitivity (§8.5)

Key results must survive reasonable parameter changes, or we report where they break. Priorities:

1. **A4 harshness** — sweep `season_amplitude` , `drought.onset_prob` , `predators.move_prob` . Report how harsh before the population collapses entirely (the boundary between "fragile" and "extinct").

2. **Movement cost (A2)** — the conservative↔explorative trade-off is most sensitive here; sweep `move_cost_per_tile` .

3. **Learning rate / rule** — `alpha` , and `recency` VS `td0` .

4. **Update scheme** — synchronous vs. sequential (flagged in §4 as able to move the A2 result).

All sweeps reuse the experiment driver with single-key config overrides, so sensitivity is the same machinery as the main runs.

Results & Calibration Narrative

This part reports what the model *does*, and — in the spirit of the spec — is candid about how the A4 result was reached, because the path is itself a finding.

A1 — the learning baseline works

A1 is a single agent, traits fixed, with only the moisture field and water sources present: a hydration-survival task. The agent must read the local moisture gradient, navigate to a water source, and drink before hydration hits zero, while energy slowly drains and `rest` recovers it.

- **It works:** the agent survives the full 1500-tick horizon across every seed tested (6/6), with no deaths.
- **It is learning, not luck:** a deliberate negative control — generating the moisture field and the water sources *independently*, so the gradient does not lead to water — kills the agent by $\approx$ tick 100 (pure thirst). Colocating water with the moisture peaks, plus a gradient-cue floor that lets a faint-but-correct gradient commit the agent to a heading, is what turns a random walk into exploitation. This was a genuine modeling bug caught and fixed during bring-up.

A2 — population and selection

A2 adds a founding population, an energy cost on movement, and birth/death. Movement cost makes the conservative↔explorative trade-off bite, and reproduction exposes the heritable disposition to selection.

- Runs produce **hundreds of births** (typ. 350–830 depending on horizon), the population sustains to the cap, and mortality is structured: deaths from movement-cost **energy** depletion and from **hydration** dominate.
- The A2 claim becomes a testable surface — ``survival ~ birthplace_endowment x exploration_trait`` — with the learning-frozen and trait-frozen ablations (shipped as config overlays) as the controls that turn the story into a result.

A3 — the full stationary ecology works

A3 layers in the temperature comfort band, vegetation, prey, and predators, all **stationary**. With the comfort band and predators now able to kill, the agent must satisfy four needs at once.

- A3 is **uniformly stable**: the population settles into the always-comfortable temperature refuge and sustains near carrying capacity (mean $\approx$ 360–383), with 0 of 2–3 seeds extinct in calibration checks.
- Mortality is dominated by comfort-band exposure and predation — the pressures the agent must learn/settle around.

A4 — fragility, and how it was calibrated

A4 toggles non-stationarity onto the *same* world: temperature oscillates seasonally, vegetation is season-gated (a seasonal famine), prey and predators move, and water sources stochastically dry and rebound. Everything an agent learned or was selected for in A3 can be invalidated. The ruling (\$9.5) was for **fragile survival** — poorly, but not total collapse — and reaching that regime took several measured iterations:

1. **First attempt → total collapse (3/3 extinct).** Diagnosis: the population booms to the cap in the fertile first half-season, then a seasonal catastrophe (cold + famine + drought) wipes it out around $t \approx 450$. Boom-bust, not fragility.
2. **Softened seasons → still collapsing.** The temperature comfort exposure was too lethal relative to how fast agents migrate to the shifting comfort band. Widening the band and easing exposure helped A3 robustness but A4 still tipped.
3. **Faster reproduction → A3 robust, A4 still 4/5 extinct.** Quicker recovery fixed A3's occasional founding failures but made A4 boom higher and crash harder: an absorbing collapse from slow recovery against sustained cold.
4. **Targeted the real mechanism.** A4's mobile predators *scatter* the herd (via fleeing) and the vegetation famine *drives foraging*, pushing agents off the always-comfortable strip into the lethal grid edges. Widening the habitable refuge, calming the predators, easing the famine, and shortening seasons into brief survivable pulses moved the system from guaranteed collapse to **precarious persistence**.

Measured A3 → A4 contrast (cap 2000 × 20 seeds, 3000 ticks)

Medians with bootstrap 95% CIs over seeds; same-seed pairing, so the contrast is the cost of non-stationarity itself. Supersedes the provisional cap-400 numbers.

Metric	A3 (stationary)	A4 (non-stationary)
extinct fraction	0.0	0.3
carrying capacity R_s	735.0 [508.0, 1043.0]	419.0 [156.0, 728.0]
max drawdown (crash severity)	0.416 [0.295, 0.601]	0.872 [0.849, 0.999]
volatility (CV)	0.05 [0.045, 0.07]	0.269 [0.14, 0.391]
Fano of per-tick deaths	1.8 [1.7, 1.9]	3.8 [2.8, 4.9]

Interpretation. A3 is uniformly stable; A4 is fragile. Surviving A4 seeds persist *precariously* — substantial populations that crash to a handful and claw back, repeatedly — while a sizeable fraction collapse outright. Individuals live roughly half as long. The physical-needs substrate **alone** cannot reliably survive non-stationarity, which is precisely the pressure Batch B must relieve.

Two honest caveats. (1) Extinction is an absorbing state, so a deeply oscillating population's extinction *rate rises with the reporting horizon*; we fix the horizon at 2500 ticks, where A4 "survives poorly but not totally," and note that this horizon-dependence *is* the fragility. (2) These are turnaround checks at $n = 2$ –5 seeds; headline rates warrant ≥ 20 seeds (the experiment driver supports exactly this). A4 sits near a tipping point, and every harshness knob is config — so "how harsh before collapse" is a dial the lab can turn, and the narrowness of the fragile band is itself a reportable result.

How to Run & Reproduce

Everything is reproducible from (code version, config, seed) . Each run writes a self-describing directory under `runs/<run_id>/` containing the per-tick stream, the per-agent lifetime summaries, the per-tick environment scalars, the fully resolved config, and a `meta.json` with the seed, environment hash, code version, and git commit.

Install

```
cd batch-a
pip install -r requirements.txt      # numpy + PyYAML (pandas optional, Parquet)
```

Run, sweep, verify

```
python -m batch_a run      --config a1.yaml --seed 0      # one run, JSON to stdout
python -m batch_a sweep    --config a2.yaml --seeds 0-9    # a seed sweep
python -m batch_a verify   --config a1.yaml --seed 0      # determinism self-check
```

Experiments with ablations and cross-seed aggregation

```
python experiments/run_experiment.py --config a2.yaml --seeds 0-19 \
  --conditions uniform,born_rich,born_poor \
  --ablations none,freeze_learning,freeze_traits
```

This aggregates across seeds with **median + percentile bootstrap CI** and writes tidy `*_runs.csv` / `*_aggregate.csv` tables. The two ablations are the controls (§1, §8.4): `freeze_learning` (does the trait alone reproduce the pattern?) and `freeze_traits` (does learning alone?). If neither alone reproduces a result, it is an **interaction** — a stronger, more honest finding.

Tests

A 22-test suite guards the invariants — determinism (same seed → identical history), config inheritance and the environment hash, toroidal field gradients and regeneration, need-vector extensibility, trait inheritance bounds, the learning on/off flag, and per-case end-to-end smoke with the §6 artifacts:

```
cd batch-a && python -m pytest tests/ -q      # 22 passed
```

Appendix A — Repository File Tree

The complete project (simulation outputs and caches omitted):

```
./gitignore
./README.md
./batch_a/__init__.py
./batch_a/__main__.py
./batch_a/agent/__init__.py
./batch_a/agent/actions.py
./batch_a/agent/agent.py
./batch_a/agent/decision.py
./batch_a/agent/learning.py
./batch_a/agent/memory.py
./batch_a/agent/perception.py
./batch_a/agent/state.py
./batch_a/agent/traits.py
./batch_a/cli.py
./batch_a/config.py
./batch_a/environment/__init__.py
./batch_a/environment/dynamics.py
./batch_a/environment/entities.py
./batch_a/environment/fields.py
./batch_a/environment/world.py
./batch_a/logging/__init__.py
./batch_a/logging/recorder.py
./batch_a/logging/schema.py
./batch_a/metrics.py
./batch_a/reproduction.py
./batch_a/rng.py
./batch_a/scheduler.py
./batch_a/sim.py
./config/a1.yaml
./config/a2.yaml
./config/a3.yaml
./config/a4.yaml
./config/ablations/a2_freeze_learning.yaml
./config/ablations/a2_freeze_traits.yaml
./config/base.yaml
./design/build_css.py
./design/tokens.json
./docs/A1_LEARNING_DECISION.md
./docs/DECISIONS.md
./docs/EXECUTION_TRACKER.md
./docs/EXPERIMENTAL_DESIGN.md
./docs/ODD.md
./docs/PARAMETERS.md
./docs/SOA_REWRITE_PLAN.md
./docs/WO2_WO3_AMENDMENT.md
./docs/build_report.py
./docs/diagnostics/a4_cap2000_analysis.json
./docs/diagnostics/a4_cap2000_read.md
./docs/diagnostics/wo1_a1_runs.csv
./docs/diagnostics/wo1_a1_summary.json
./docs/diagnostics/wo2_collinearity.csv
./docs/diagnostics/wo2_collinearity.json
./docs/diagnostics/wo2_collinearity_plane.svg
./docs/diagnostics/wo2_crashrobust_pilot.csv
./docs/diagnostics/wo2_crashrobust_pilot.json
./docs/diagnostics/wo2_crashrobust_read.md
./docs/diagnostics/wo2_full20/wo2_a3_runs.csv
./docs/diagnostics/wo2_full20/wo2_a3_summary.json
./docs/diagnostics/wo2_full20/wo2_checkpoint.jsonl
./docs/diagnostics/wo2_full20/wo2_full20_analysis.json
./docs/diagnostics/wo2_full20/wo2_full20_read.md
./docs/diagnostics/wo2_pilot_uniform_none_8seed.jsonl
./docs/diagnostics/wo3_full20/wo3_checkpoint.jsonl
./docs/diagnostics/wo3_full20/wo3_runs.csv
./docs/diagnostics/wo3_full20/wo3_summary.json
```

```
./docs/diagnostics/wo4_a3_cap_runs.csv
./docs/diagnostics/wo4_a3_cap_summary.json
./docs/report_content.py
./experiments/analyze_a4_cap2000.py
./experiments/analyze_wo2_crashrobust.py
./experiments/analyze_wo2_full20.py
./experiments/check_wo2_collinearity.py
./experiments/grind_status.py
./experiments/results/wo1_a1_runs.csv
./experiments/results/wo1_a1_summary.json
./experiments/results/wo2_a3_runs.csv
./experiments/results/wo2_a3_summary.json
./experiments/results/wo2_checkpoint.jsonl
./experiments/results/wo4_a3_cap_runs.csv
./experiments/results/wo4_a3_cap_summary.json
./experiments/results/wo4_checkpoint.jsonl
./experiments/rng_trace.py
./experiments/run_experiment.py
./experiments/run_wo2_grind.sh
./experiments/run_wo3_grind.sh
./experiments/verify_perception.py
./experiments/wo1_a1_diagnostic.py
./experiments/wo2_a3_birthplace.py
./experiments/wo3_a4_mortality.py
./experiments/wo4_a3_cap.py
./pyproject.toml
./requirements.txt
./tests/conftest.py
./tests/test_agent.py
./tests/test_config.py
./tests/test_design_tokens.py
./tests/test_determinism.py
./tests/test_environment.py
./tests/test_perception_equivalence.py
./tests/test_rng_trace.py
./tests/test_smoke.py
./tests/test_wo_instruments.py
./viz/__init__.py
./viz/build.py
./viz/export_playback.py
./viz/figures/__init__.py
./viz/figures/a1_learning_gradient.py
./viz/figures/wo2_selection_moderator.py
./viz/figures/wo3_mortality_structure.py
./viz/style.py
```

Appendix B — Configuration Listings

Config is the single source of truth; per-case files override only the deltas they name. Listings are the committed values.

base.yaml — all defaults

```
# =====
# Batch A — base configuration (all defaults live here).
# Per-case configs (al..a4) and ablations override only the deltas they change.
# NO magic numbers anywhere else in the code: every constant is here, with units.
# See docs/PARAMETERS.md for the human-readable parameter table.
# =====

case: base

run:
  max_ticks: 2000          # ticks; a run also ends early on extinction
  stop_on_extinction: true # end the run when no agents are alive

# -----
# World / environment (§2)
# -----
world:
  size: 64          # grid is size x size tiles (parameterize up to 256)
  boundary: toroidal # 'toroidal' (wrap) or 'walls'
  perception_radius: 2 # bounded Moore radius r in {1,2}; local info only

# How local field gradients become movement cues (perception.py). A gradient
# direction yields a cue of strength clamp(floor + magnitude*gain, .., 1). The
# floor lets a faint-but-correct gradient still commit the agent to a heading,
# which is what makes long-range "follow moisture to water" navigation work.
perception:
  impl: vectorized          # bit-identical to 'scalar' at full horizon (verify_perception, all 4 cases);
                           # 'scalar' stays selectable as the reference for the diff harness.
  gradient_cue_gain: 8.0
  gradient_cue_floor: 0.35

# Continuous scalar fields (§2.1). Each is generated once and (unless a case
# enables dynamics) held fixed across all comparisons.
fields:
  moisture:
    enabled: true
    clusters: 5          # number of gaussian "wet" clusters
    cluster_sigma: 6.0    # tiles; spread of each cluster
    baseline: 0.05        # field floor [0,1]
  temperature:
    enabled: false        # introduced in A3
    gradient_axis: y       # 'x' or 'y'; linear gradient across the grid
    low: 0.0              # field value at one edge [0,1]
    high: 1.0             # field value at the other edge [0,1]
    comfort_low: 0.25      # comfort band lower bound [0,1]
    comfort_high: 0.75     # comfort band upper bound [0,1]
    season_amplitude: 0.0  # A4: peak seasonal offset added to the field [0,1]
    season_period: 365     # ticks per full seasonal cycle (A4)
  risk:
    enabled: false        # derived from predators when they exist (A3)
    predator_sigma: 4.0   # tiles; spread of each predator's risk halo
    baseline: 0.0         # ambient risk floor [0,1]

# Discrete consumable / threat entities (§2.2). Quantities live on a per-tile
# grid; sources sit on a fixed clustered mask and regenerate toward a cap.
entities:
  water:
    enabled: true
    introduced_in: A1
    sources: 6            # number of discrete water source clusters
```

```

cluster_sigma: 2.0      # tiles
capacity: 1.0           # max stored quantity per source tile [0,1]
regen_per_tick: 0.05    # quantity restored per tick toward capacity
restores: hydration     # which need drinking restores
action: drink
efficiency: 1.0         # fraction of consumed quantity converted to need gain
vegetation:
  enabled: false        # A3
  introduced_in: A3
  patches: 8
  cluster_sigma: 3.0
  capacity: 1.0
  regen_per_tick: 0.02
  restores: energy
  action: eat
  efficiency: 0.8
  season_gated: false   # A4: edible only in-season
prey:
  enabled: false        # A3
  introduced_in: A3
  count: 12             # number of discrete prey
  capacity: 1.0
  regen_per_tick: 0.0    # prey do not regrow in place; they respawn
  respawn_ticks: 80     # ticks after consumption before a prey respawns
  restores: energy
  action: eat
  efficiency: 1.0
  mobile: false         # A4: prey move
  move_prob: 0.0        # A4: per-tick probability a prey steps
predators:
  enabled: false        # A3
  introduced_in: A3
  count: 6
  attack_radius: 1      # tiles; predation possible within this Chebyshev range
  predation_prob: 0.15  # per-tick kill probability inside attack radius
  mobile: false         # A4: predators move
  move_prob: 0.0        # A4: per-tick probability a predator steps

# -----
# Non-stationarity (§2.2, A4). All inert unless a case turns them on.
# -----
nonstationarity:
  seasonality: false     # temperature oscillates; vegetation season-gated
  mobile_fauna: false    # prey & predators move
drought:
  enabled: false
  onset_prob: 0.002      # per-tick probability a wet period flips to drought
  recovery_prob: 0.01    # per-tick probability a drought flips back to wet
  dry_regen_factor: 0.1  # water regen multiplier while in drought

# -----
# Agent (§3)
# -----
# Homeostatic need substrate (§3.1). Order is preserved; Batch B appends needs.
# drain_per_tick: endogenous drain. target/threshold in [0,1], threshold < target.
needs:
  energy:
    initial: 1.0
    target: 1.0
    threshold: 0.25
    drain_per_tick: 0.004
  hydration:
    initial: 1.0
    target: 1.0
    threshold: 0.25
    drain_per_tick: 0.010
  temperature_comfort:
    initial: 1.0
    target: 1.0
    threshold: 0.20
    drain_per_tick: 0.000 # only drains via exposure outside comfort band (A3)
    exposure_gain: 0.15   # drain = exposure_gain * distance_outside_band
    recovery: 0.05        # restored per tick while inside the comfort band

```

```

safety:
  initial: 1.0
  target: 1.0
  threshold: 0.20
  drain_per_tick: 0.000 # only drains via risk exposure (A3)
  exposure_gain: 0.50 # drain = exposure_gain * local_risk
  recovery: 0.05 # restored per tick while local risk is low

# Action effects (§3.3). The action set is a registry (see actions.py).
actions:
  rest:
    restores: energy
    amount: 0.06 # need gain per tick of resting
  move_cost_per_tile: 0.0 # energy drained per tile moved (A2 sets this > 0)
  flee_cost_per_tile: 0.0 # energy drained per tile fled (tracks move cost in A2)

# Decision rule (§3.4):  $U(a) = \sum_i w_i(t) \cdot Q_i(a) - \text{MoveCost}(a) - \text{Risk}(a)$ 
decision:
  weighting: deficit_proportional # the single swappable weight function
  epsilon: 0.05 # base  $\epsilon$  for  $\epsilon$ -stochastic action choice
  risk_weight: 1.0 # coefficient on Risk(a) in U(a)
  move_cost_weight: 1.0 # coefficient on MoveCost(a) in U(a)

# Within-lifetime learning (§3.5). One flag disables it for ablations.
learning:
  enabled: true
  rule: recency # 'recency' (delta rule) or 'td0'
  alpha: 0.2 # learning rate
  gamma: 0.9 # discount (td0 only)
  gain_init: 1.0 # initial learned gain on each (need, action) affordance
  gain_min: 0.1
  gain_max: 3.0

# Heritable traits (§3.6). One flag fixes all traits identical for ablations.
traits:
  fix_identical: false # ablation: everyone gets the same disposition
  exploration:
    init_mean: 0.5 # mean exploration disposition at world seeding [0,1]
    init_sd: 0.15 # sd of initial disposition across the founder pop
    mutation_sd: 0.05 # gaussian mutation sd passed to offspring
    epsilon_scale: 0.30 #  $\epsilon$  contribution = epsilon + exploration*epsilon_scale
    step_noise_scale: 0.50 # prob. of a random (non-greedy) move step

# Reproduction (§9.4: asexual clone-with-mutation). Inert when disabled (A1).
reproduction:
  enabled: false # A2+ enable
  maturity_age: 40 # ticks before an agent can reproduce
  cooldown: 25 # ticks between births for one agent
  readiness_min_need: 0.6 # all needs must be >= this to reproduce
  energy_cost: 0.4 # energy spent producing one offspring
  offspring_spawn_radius: 1 # tiles from parent where offspring appears
  max_population: 400 # hard cap to keep runs bounded

# -----
# Initialization (§5). Birthplace is the independent variable, not incidental.
# -----
init:
  n_agents: 1 # founder population (A1 = single agent)
  spawn_mode: uniform # 'uniform' | 'born_rich' | 'born_poor'
  endowment_percentile: 0.85 # for born_rich: spawn among tiles above this pct
  # for born_poor: spawn among tiles below (1 - this)
  endowment_radius: 2 # radius r used to compute birthplace endowment scalar

# -----
# Logging (§6)
# -----
logging:
  format: csv # 'csv' (always available) or 'parquet' (needs pandas)
  per_tick: true # write the per-tick per-agent stream
  out_dir: runs # runs/<run_id>/ holds all artifacts for a run

# -----
# Metrics / operational definitions (§7)

```

```
# -----
metrics:
  settle_displacement_threshold: 1.5 # tiles; below this counts as "settled"
  settle_window: 50                 # ticks displacement must stay low to settle
  resource_percentile_k: 0.75        # Kth percentile defining a "good" tile
```

a1.yaml

```
# A1 – single-agent learning baseline (§1). No reproduction, traits fixed.
# Only the moisture field + water sources exist. The task is hydration survival:
# navigate the moisture gradient to water, drink before hydration hits zero,
# while energy slowly drains and rest recovers it.
case: a1

run:
  max_ticks: 1500

init:
  n_agents: 1
  spawn_mode: uniform

reproduction:
  enabled: false

traits:
  fix_identical: true      # A1 holds traits fixed; learning is the only adaptation

learning:
  enabled: true
```

a2.yaml

```
# A2 – population + movement cost + birth/death (§1, §2.2). Selection turns on.
# This is where the birthplace insights live: "born on water", "born poor".
# Movement now drains energy, so the conservative<->explorative trade-off bites.
case: a2

run:
  max_ticks: 3000

init:
  n_agents: 40
  spawn_mode: uniform      # sweep over uniform | born_rich | born_poor

actions:
  move_cost_per_tile: 0.012
  flee_cost_per_tile: 0.012

reproduction:
  enabled: true

traits:
  fix_identical: false     # heritable exploration disposition now varies & is selected

learning:
  enabled: true
```

a3.yaml

```
# A3 – full stationary ecology (§2.2). Adds heat (comfort band), vegetation,
# prey, and predators (risk). Everything still STATIONARY: fauna don't move,
# seasons don't turn. This is the world A4 will invalidate.
case: a3

run:
  max_ticks: 3000
```

```

init:
  n_agents: 40
  spawn_mode: uniform

actions:
  move_cost_per_tile: 0.012
  flee_cost_per_tile: 0.012

fields:
  temperature:
    enabled: true
  risk:
    enabled: true

entities:
  vegetation:
    enabled: true
  prey:
    enabled: true
    mobile: false
  predators:
    enabled: true
    mobile: false

reproduction:
  enabled: true

traits:
  fix_identical: false

learning:
  enabled: true

```

a4.yaml

```

# A4 – the stationary -> non-stationary phase change (§2.2, §9.5).
# Seasons turn, fauna move, water sources dry and rebound. Everything an agent
# learned or was selected for in A3 can be invalidated here.
#
# Harshness target: FRAGILE survival (per §9.5 ruling) – the population persists
# but poorly, leaving pressure for Batch B's mechanisms to act on. Not total
# collapse. All harshness knobs are here so it can be re-tuned empirically.
case: a4

run:
  max_ticks: 2500          # reporting horizon for "fragile survival"; extending it
                           # reveals the population is non-viable long-term (the
                           # quantitative face of fragility – see docs §A4)

init:
  n_agents: 60
  spawn_mode: uniform

actions:
  move_cost_per_tile: 0.012
  flee_cost_per_tile: 0.012

fields:
  temperature:
    enabled: true
    season_amplitude: 0.05 # seasonal swing periodically narrows the comfort band
    season_period: 400
  risk:
    enabled: true

entities:
  vegetation:
    enabled: true
    season_gated: true      # edible only part of the year -> seasonal famine
    regen_per_tick: 0.03    # slightly richer so the famine half-cycle isn't fatal
  prey:

```

```

    enabled: true
    mobile: true
    move_prob: 0.3
predators:
    enabled: true
    mobile: true
    move_prob: 0.15      # roam, but scatter the herd less violently
    count: 4

nonstationarity:
    seasonality: true
    mobile_fauna: true
    drought:
        enabled: true
        onset_prob: 0.0018    # fragile, not catastrophic
        recovery_prob: 0.012
        dry_regen_factor: 0.30

reproduction:
    enabled: true

traits:
    fix_identical: false

learning:
    enabled: true

```

ablations/a2_freeze_learning.yaml

```

# Ablation (§1, §8.4): freeze learning. Does the trait alone reproduce the
# A2 survival pattern? Q-values never update; only heritable disposition adapts.
extends: a2.yaml
case: a2_freeze_learning
learning:
    enabled: false

```

ablations/a2_freeze_traits.yaml

```

# Ablation (§1, §8.4): freeze traits. Does learning alone reproduce the A2
# survival pattern? Everyone gets an identical disposition (no selection on the
# exploration trait); only within-lifetime learning adapts.
extends: a2.yaml
case: a2_freeze_traits
traits:
    fix_identical: true
learning:
    enabled: true

```